EdgeML26 - Course Project Report

Team Members

Matin Moradi - Student Number: 2503299
Tariq Aziz - Student Number: 2503275

GitHub Repository:

https://github.com/it-teaching-abo-akademi/course-project-report-matinmoradi80

May 2026
Abo Akademi University

Keyword Spotting Project

1. Introduction

In this project, we describe the design and implementation of a keyword spotting system using an Artificial Neural Network (ANN) deployed on an Arduino Nano 33 BLE Sense embedded board. The goal of the project was to spot city names (Oslo and Tallin) in a phrase like “weather in Oslo”.

The following hardware and software were using during this project:

- Hardware: Arduino Nano 33 BLE Sense (Cortex-M4, built-in microphone)
- Platform: Edge Impulse (https://studio.edgeimpulse.com)
- Operating System: Windows 11
- Feature Extraction: Mel-Frequency Cepstral Coefficients (MFCC)
- Learning Block: Classification (Keras)

2. Performed Tasks

2.1 Environment Setup

We used official Edge Impulse documentation and instructions provided to configure the development environment for Arduino Nano 33 BLE Sense. Given below are the list of steps performed:

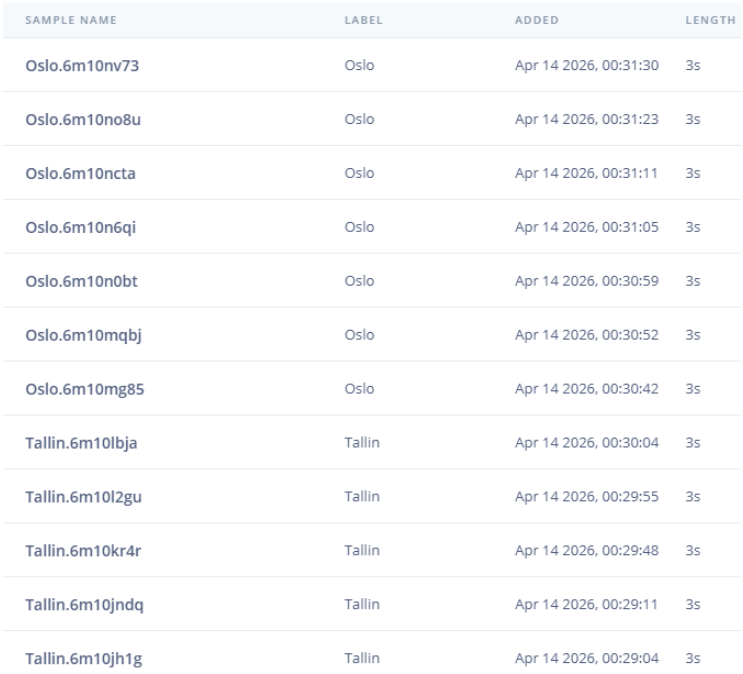
1. Node.js was installed as a prerequisite for Edge Impulse CLI.
2. Arduino CLI was installed and added to the system PATH.
3. The arduino:mbed_nano board package was installed.
4. Edge Impulse CLI was installed using npm and PATH was configured.
5. Edge Impulse firmware was flashed onto the Nano 33 BLE Sense.
6. The Arduino board was connected to Edge Impulse using edge-impulse-daemon with an API key.



Figure 1: Arduino Nano 33 BLE Sense connected to Edge Impulse project

2.2 Data Collection

The audio data was collected using the built-in microphone on the Arduino Nano 33 BLE Sense Board and Android phone. Arduino Nano was connected to the edge impulse for direct streaming of audio samples. We (Matin & Tariq) recorded the phrases ‘weather in Tallin’, and ‘weather in Oslo’ in our voices. We tried changing our accent and speed for better variation in the dataset. Moreover, two further classes - noise and unknown were added to the dataset to improve the performance of model. These samples were either downloaded or recorded via text-to-speech websites.



SAMPLE NAME	LABEL	ADDED	LENGTH
Oslo.6m10nv73	Oslo	Apr 14 2026, 00:31:30	3s
Oslo.6m10no8u	Oslo	Apr 14 2026, 00:31:23	3s
Oslo.6m10ncta	Oslo	Apr 14 2026, 00:31:11	3s
Oslo.6m10n6qi	Oslo	Apr 14 2026, 00:31:05	3s
Oslo.6m10n0bt	Oslo	Apr 14 2026, 00:30:59	3s
Oslo.6m10mqbj	Oslo	Apr 14 2026, 00:30:52	3s
Oslo.6m10mg85	Oslo	Apr 14 2026, 00:30:42	3s
Tallin.6m10lbja	Tallin	Apr 14 2026, 00:30:04	3s
Tallin.6m10l2gu	Tallin	Apr 14 2026, 00:29:55	3s
Tallin.6m10kr4r	Tallin	Apr 14 2026, 00:29:48	3s
Tallin.6m10jndq	Tallin	Apr 14 2026, 00:29:11	3s
Tallin.6m10jh1g	Tallin	Apr 14 2026, 00:29:04	3s

Figure 2: Data acquisition page showing training samples

Dataset Summary:

Label	**# Samples**	**Total Duration**
Oslo	272	9m 59s
Tallin	274	10m 21s
Noise	719	9m 34s
Unknown	720	9m 40s

2.3 Task Division

We divided the tasks among ourselves as follows:

Task	Description	Responsible
Environment Setup	Install Arduino CLI, Edge Impulse CLI, connect board	Matin & Tariq
Data Collection	Record audio samples for Oslo and Tallinn	Matin & Tariq
Dataset Augmentation	Add noise/unknown samples, text-to-speech generation	Matin
Model Training & Testing	Configure impulse, MFCC features, NN classifier, Model testing, compare experiments	Tariq
Report Writing	Document all experiments, write conclusions	Matin & Tariq

2.4 Model Design

The built-in model in Edge Impulse was employed for training with the following configurations:

- Window size: 1000ms, Window stride: 500ms, Frequency: 16000 Hz
- Processing block: Audio (MFCC) - 13 coefficients, Frame length 0.025, Frame stride 0.02
- Learning block: Classification (Keras) - 2x Conv1D layers with Dropout
- Output: 4 classes (Oslo, Tallinn, noise, unknown)

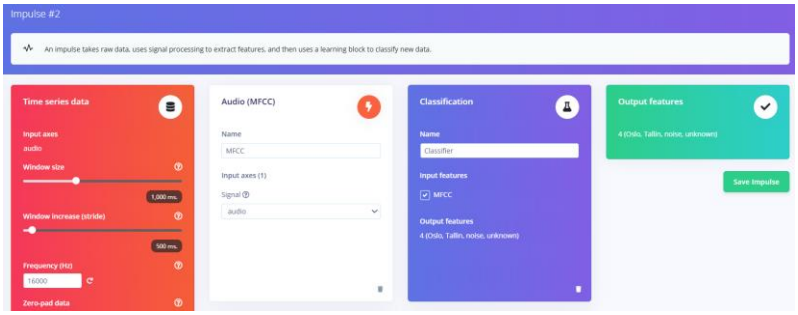


Figure 3: Impulse for model training

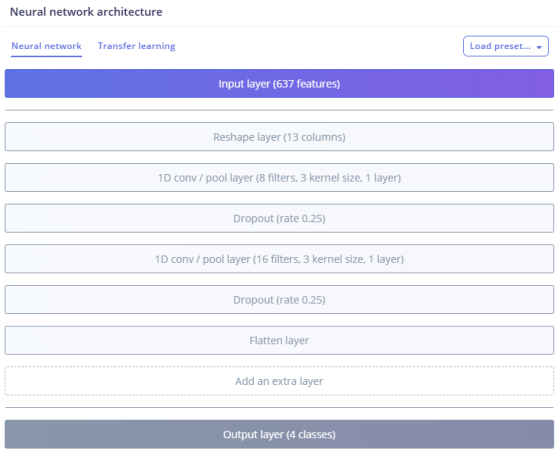


Figure 4: Neural network architecture for model training

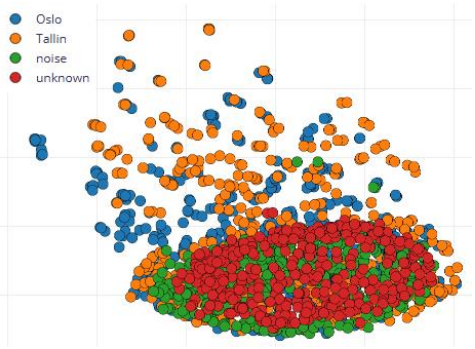
3. Results

3.1 Feature Extraction

The following set of parameters were used to extract MFCC features from the audio samples.

- Number of coefficients: 13
- Frame length: 0.025
- Frame stride: 0.02
- Filter number: 32
- FFT length: 512
- Normalization window size: 151
- Low frequency: 51

The Feature Explorer of the Edge Impulse shows the clustering of dataset.



Feature explorer shows that the clusters for unknown and noise are densely packed and close to each other. On the other hand, clusters for our target cities are scattered and not densely packed.

3.2 Training Results

We experimented with varying settings and dataset to improve the performance of our model. We summarized the results of our experiments in the table below.

# Exp	Changes Made	Accuracy
1	Default settings, no augmentation, on two classes	69%
2	Model trained on 4 classes, unknown and noise added	76%
3	Model trained on 4 classes and optimized configuration	80%



Figure 5: Confusion matrix for our trained model

Key observations from confusion matrix:

- Overall accuracy: 79.9%
- Oslo correctly classified: 75.7% - 19.3% confused with Tallinn
- Tallinn correctly classified: 72.4% - 24.6% confused with Oslo
- Noise: 95%, Unknown: 84.5% - both performing well

Confusion matrix shows that the model performs exceptionally well on capturing noise and unknown signals. The reason for performing well on noise and unknown is that the length of audio signal for these classes is 1s long, while cities audio signals are 3s long.

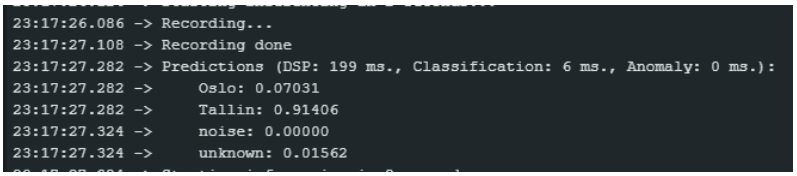
3.3 Model Testing and Deployment

The trained model was later tested on the held-out test dataset.



Figure 6: Model's performance on test data

Then we exported the package from Edge Impulse and deployed it on the board. This is an example output:



4. Discussion and Conclusions

4.1 Interpretation of Results

Our final model achieved an accuracy of 79.9% for a four-class keyword spotting system. It misclassified few signals for Oslo and Tallin. Some of the likely reasons for misclassification can be:

- Duration of 3s, which is way too long for a short phrase.
- Limited accent diversity in training data.
- Both words have similar phonetic structure – short, two-syllable words

4.2 Proposed Improvements

Based on our experimentation, we proposed few ideas for future work.

- Collect more diverse data with different accents and speed.
- Use of text-to-speech tools for better data quality and diversity.
- Use of different learning block like transfer learning instead of standard classifier.

5. Reflection

The keyword spotting project introduced us to a complete pipeline of embedded machine learning. Before this course, we were mostly working on theoretical machine learning. This course gave us an opportunity to design, train, and deploy machine learning model directly on the hardware platform with limited computational resources. We also came across new concepts such as MFCC for feature extraction from audio signals. The most surprising aspect of the project was to witness how data quality improved the performance of the model. A general assumption is to have as much data as possible. However, we found that data quality matters more than having a bunch of data for training. Thus, adding diverse data, and trying different accents improved the performance of our model. The most challenging part of the project was achieving good accuracy for our model. Our model initially struggled with just two classes only. Later, we added noise and unknown classes and also added more data with different accents and speed. This improved the performance of our model. The most satisfying part of the project was the iterative process of running the experiments and observing improvement in the performance of model. This gave us a better idea of experiment with audio signals and what approach to follow in such scenarios. Completing the full cycle from data acquisition to model deployment and then inferring live was a satisfying aspect of the project.

Rock-Paper-Scissors Game

1. Introduction

The goal of this project was to deploy a TensorFlow image classification model on an Arduino Nano 33 BLE Sense, without using any high-level platform such as Edge Impulse. The model classifies hand gestures from the rock-paper-scissors dataset. The full pipeline involves capturing a photo with the OV7670 camera module connected to the board, preprocessing the image, running inference using TensorFlow Lite Micro, and printing the predicted class.

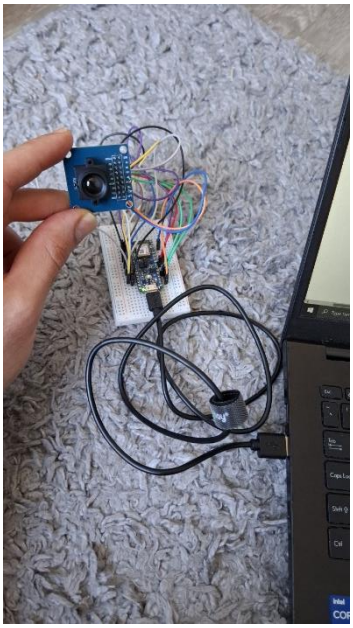
The first step was the environment setup, library installation and basic camera testing. Then it was training the model and applying compression techniques (pruning and quantization) in TensorFlow. The third session combined everything by writing the Arduino sketch that loads the compressed model and produces predictions. We used the Arduino IDE for compiling and uploading code and Google Colab for training and compressing the model.

2. Performed Tasks

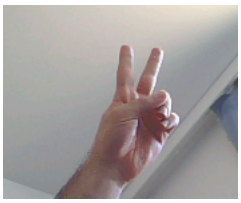
2.1 Environment Setup and Camera Testing

We first installed the TensorFlow Lite Micro library for Arduino and then cloned the EdgeML_26 repository that contains the notebooks and scripts needed for the project. The Arduino_OV767X library was installed from the Arduino Library Manager so the IDE could talk to the camera module.

The next step was connecting the camera to the board which needed focus and time because there are 16 wires and a single wrong pin causes the camera examples to fail.



Once we double-checked the wiring against the library's pinout, the Camera Capture example sketch worked and the captured frame was printed as a hex string on the serial monitor and then we used the DisplayRawImage script to visualize the captured frame in Python. These are some initial test images captured:




The resolution of these initial test images is QCIF which is 176x144. We also tried with a different resolution. With higher resolutions we faced error so we tried QQVGA which is 160x120 and smaller than the initial resolution:



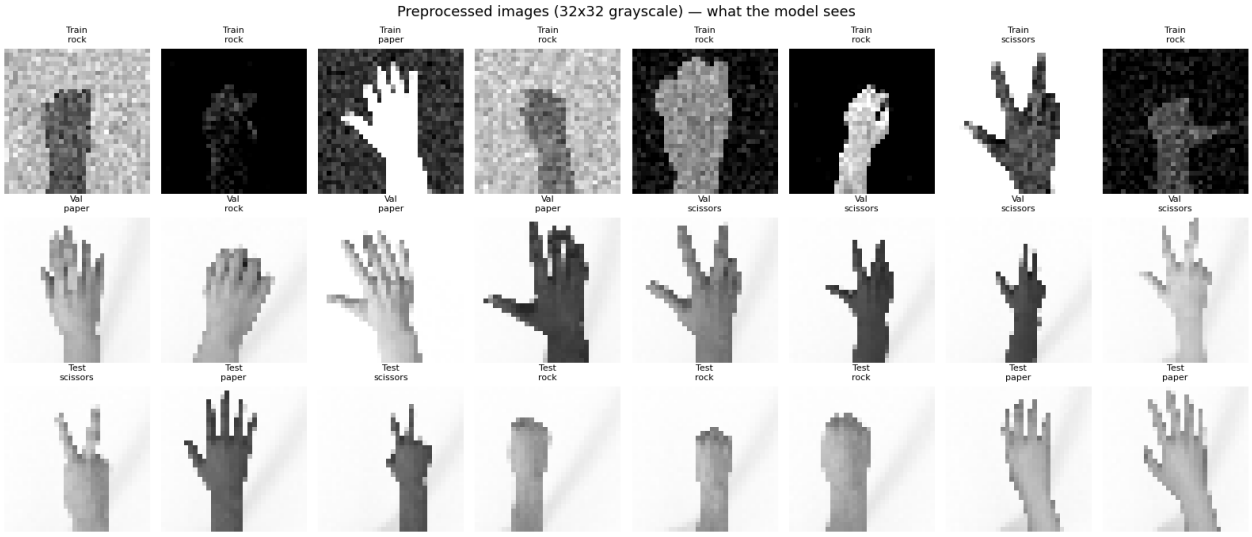
We then tried to capture grayscale by changing some few parts of the arduino and python code (QCFI resolution):




2.2 Model Training and Compression

For this phase we used the provided notebook on Google Colab. We loaded the rock_paper_scissors dataset from TensorFlow Datasets where a full model is trained and then resized to fit on the board.

The preprocessing and training parts were already in the notebook but we did some improvements ourselves too. For example, by default in the notebook, all images were resized from the original 300x300 RGB down to 32x32, converted to grayscale, and normalized to float32 in the range [0, 1]. However, to make the model more robust to the conditions it would see through the camera, we added an augmentation pipeline that runs only on the training data. It applies random brightness changes, random contrast, random horizontal flips, occasional full color inversion, and gaussian noise. The pixel values are then mapped to [0, 1] after augmentation. We added this change because the CGI hands are always on a white background, but real camera frames may have darker backgrounds or be noisy, and training the model on inverted images makes it more robust:



The architecture is a CNN with three convolutional blocks which then feeds into a dense layer, and finally a softmax output over the three classes. However we did some changes in the training process. Since we made the training set more challenging, we decreased the learning rate to 0.0005 and increased the number of epochs to 15 for better results and generalization.

After training the main big model, we tried the compression methods, pruning and quantization. Pruning means setting some weights to zero so the model becomes sparser and easier to compress. We tried two approaches. Task 1A prunes the whole model gradually, from 0% zeros up to 50%. Task 1B prunes only the convolutional layers and keeps them at a fixed 50% zeros.

Quantization stores the model's numbers as 8-bit integers instead of 32-bit floats, which makes things smaller and faster. We tried three different post-training quantization methods. Task 2A converts only the weights. Task 2B converts both weights and activations but keeps input and output as float32, using 100 calibration images to pick the right scales. Task 2C is the same but forces input and output to uint8. We deployed 2C because the camera already gives us uint8 pixels so no float conversion is needed.

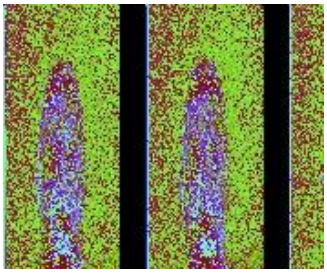
In the end, the .tflite file was converted to .h and placed in the arduino sketch folder.

2.4 Deployment

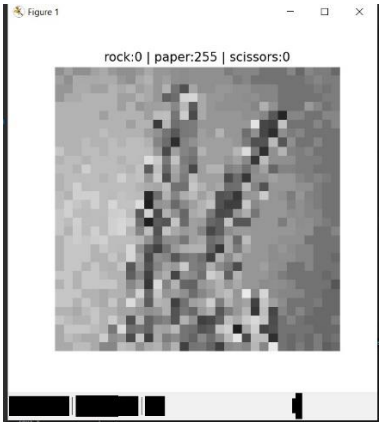
The last step was to write an Arduino script to capture the frames and run the inference and deploy it on the board. It has two functions: setup() and loop(). setup() initializes the camera, loads the model, and prepares the interpreter with a 40 KB tensor arena. loop() captures a frame every three seconds, resizes it to 32x32 grayscale, runs inference, and prints the class with the highest score.

Since in the serial monitor, it only shows the inference numbers we need to have a code to display the captured image, so that we can place our hand in a better location in front of the camera and verify if the captured frame is as expected. Therefore, we use a python script that decodes the 32x32 frames from the Arduino's serial output and shows them live while saving each as a PNG so we can debug more easily.

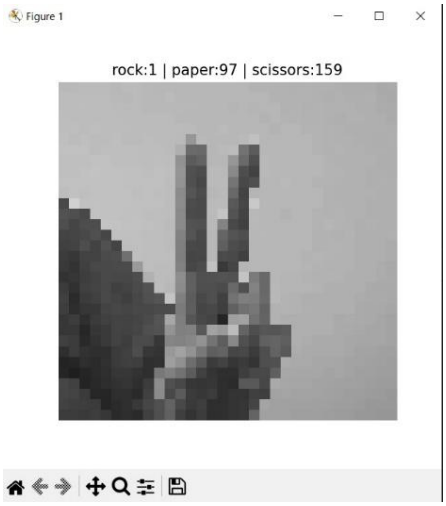
This stage was a bit challenging because initially the results were not good and we realized the images captured are not correct because the connectivity issues, wires being loose, and object moving in front of the camera:



After fixing the issues, we could saw the image but still with a lot of noise and distortion:



The problem was that the image decoding pipeline had incorrect byte ordering and not precise channel conversion, which caused the model to receive corrupted pixel data. Fixing the byte swap, using more accurate integer arithmetic for RGB-to-grayscale conversion, and adding a check for empty frames fixed the issue and produced clean input images for the model:



We generated three .h files with different combinations of compression methods, deployed them on the board and compared their accuracy:

1. **model_baseline.h**: baseline trained model + full int8 quantization. Accuracy 0.9247.
2. **model_pruned_full.h**: Task 1A pruned model + full int8 quantization. Accuracy 0.9086.
3. **model_pruned_conv2d.h**: Task 1B pruned model + full int8 quantization. Accuracy 0.8468.

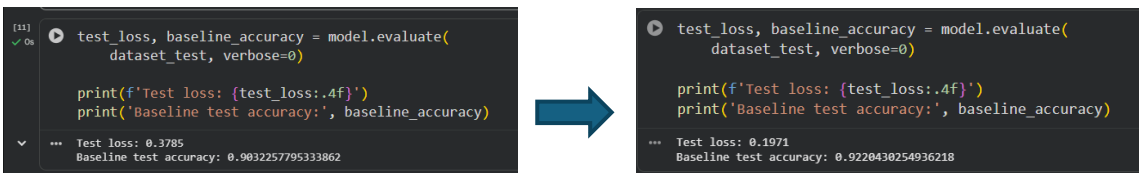
2.5 Task Division

We divided the tasks among ourselves as follows:

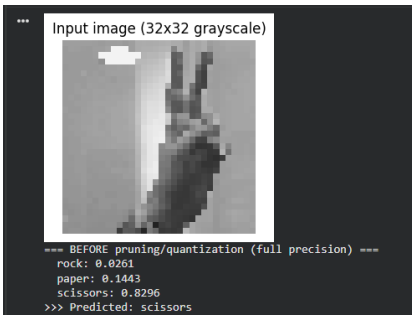
Task	Description	Responsible
Environment Setup and Camera Testing	Installing libraries and connecting the camera wires	Matin & Tariq
model training	Data augmentation, improving the training process, and test the accuracy	Tariq
Pruning	Implementing the pruning compression method	Tariq
Quantization	Implementing the quantization compression method	Matin
Writing the Arduino script	writing and testing the Arduino code with the static image	Tariq
Deployment	Debugging, deploying, testing, and accuracy improvement	Matin
Report Writing	Document all experiments, write conclusions	Matin & Tariq

3. Results

The accuracy of the default full-precision model was 90.3%. After the improvements and data augmentation, we got 92.2% accuracy.



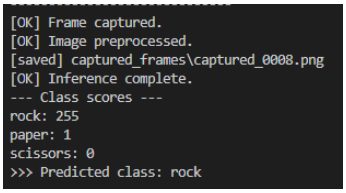
A sample inference can be seen here:

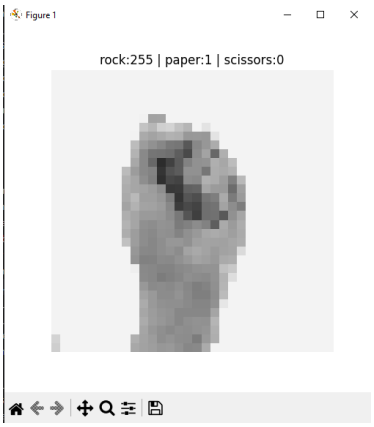


Following is the result of different compression methods and how they affected the size and the accuracy of the model:

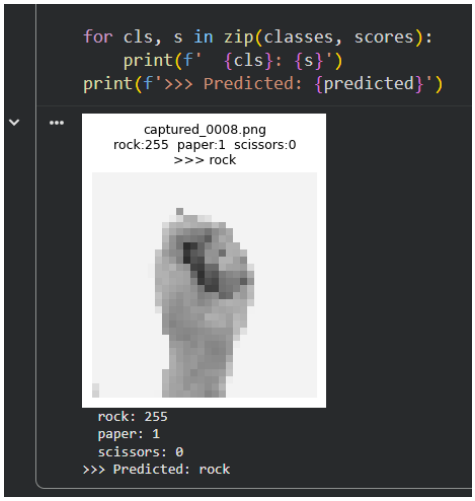
Variant	Size (bytes)	Test acc
baseline (float32 .h5)	122,736	0.9220
pruned full-model (float32 .h5)	122,736	0.9113
pruned conv2d-only (float32 .h5)	122,736	0.8575
baseline + dynamic range quant	30,840	0.9220
baseline + int quant (float I/O)	32,720	0.9247
baseline + full int quant (uint8)	32,736	0.9247
pruned_full + full int quant (uint8)	32,736	0.9086
pruned_conv2d + full int quant (uint8)	32,736	0.8468

We also tried to validate the results taken from the camera and inferenced on device vs. the results from the inference by the python code. This is an example where you can see the inference on the device on the left, and the python inference on the right (model_pruned_full.h):









And finally, some screenshots from the live inference on the device (model_baseline.h):



4. Discussion and Conclusions

4.1 Interpretation of Results

On the baseline full-precision model we got 92% test accuracy. When we tried pruning, the accuracy dropped a bit for the whole-model pruning (around 91%) and dropped more for the Conv2D-only pruning (around 86%). But the size didn't drop. So pruning was not very effective in our case.

Quantization worked much better. Full integer quantization with uint8 input and output made the model about 4 times smaller (from around 122 KB down to around 33 KB) and the accuracy stayed almost the same. The uint8 type also fits well with the camera, because the grayscale pixels we send into the model are already uint8.

When we combined pruning with quantization, the size was the same as just quantization but the accuracy was lower, because the lower accuracy from pruning stayed in the model. So the best choice for our deployment was the baseline model with full integer quantization and no pruning.

We also checked that the model gives the same answers on the Arduino and in Python. While running the inference on the device, we also save the captured frame in a folder where we later give it to the local python file to run the inference independently. The scores match, so we know the model behaves the same way on the board as it does on the computer.

4.2 Proposed Improvements

One limitation was the training data. It was artificial images on a white background, while the camera sees real hands and real backgrounds. We added a data augmentation and it helped, but there's still room for improvement. One improvement would be to collect some real images with the camera and fine-tune the model on them.

Also we can try:

- Try RGB images instead of grayscale
- Define a threshold so that we only print the prediction if we're more certain
- Define a fourth class like 'none' so we don't detect those classes when for example there's no hand in front of the camera

5. Reflection

This project was interesting because, unlike the first project, we had to train and compress the model ourselves instead of working with an easy UI like Edge Impulse. We also had to connect and use the camera, which was both challenging and satisfying. We learned how different the results can be from your expectations once you actually deploy and test the model on the board. It works in the Python code, but when you try to deploy it, you face compilation errors, corrupted pixels, and low accuracy results that you have to resolve one by one. So we would say the most challenging part is always the deployment and testing, which is actually an improvement and deployment cycle.
The most enjoyable moment was connecting the camera to the board and finally capturing the image. It was the first time in a while that we used our hands to build something physically, instead of using them to code by typing, and it felt interesting to see our faces show up in the camera. Another satisfying moment was fixing the bug in image capture. That was the bottleneck, and once it was solved, we could finally see the correct inferences showing up on the screen.